Design and Deployment of a Containerized Microservices System with Terraform-Based Infrastructure Provisioning and Incident Response Simulation

Student: Miras Aliyev

Group: SE-2427

Project Overview

This project implements a containerized microservices application with a frontend, API gateway, five FastAPI services, PostgreSQL, Prometheus, Grafana, and Terraform infrastructure files. It also includes a realistic Order Service incident simulation, response process, and postmortem analysis.

Core Components

- Frontend Layer: JavaScript application served by Nginx.
- API Gateway: Nginx reverse proxy for service routing.
- Microservices: Authentication, User, Product, Order, and Chat services.
- Database Layer: PostgreSQL container.
- Monitoring Layer: Prometheus metrics collection and Grafana visualization.
- Infrastructure Layer: Terraform configuration for AWS EC2 provisioning.

Functional And Non-Functional Coverage

- The frontend provides login, product retrieval, order creation, chat, and service status views.
- The Authentication Service validates user credentials and returns a JWT token.
- The Product Service returns product data and supports product creation.
- The Order Service creates orders and updates product stock in PostgreSQL.
- Each backend service exposes health and metrics endpoints.
- Docker Compose provides isolated containers and internal networking.
- Prometheus collects service metrics and Grafana visualizes availability and request rate.
- Fault isolation is demonstrated because the incident affects the Order Service while other services remain available.

Deployment Steps

1. Open PowerShell in the project folder.
2. Run: docker compose up --build -d
3. Open http://localhost for the frontend.
4. Open http://localhost:9090 for Prometheus.
5. Open http://localhost:3000 for Grafana using admin / admin123.
6. Validate containers with docker compose ps.

Terraform Implementation

The Terraform configuration provisions an AWS VPC, public subnet, internet gateway, public route table, security group, and EC2 instance. Network access rules open SSH on port 22, HTTP on port 80, Grafana on port 3000, and Prometheus on port 9090. The outputs file returns the instance public IP and URLs for the frontend, Grafana, and Prometheus.

Terraform Commands

```
cd terraform
terraform init
terraform plan
terraform apply
```

Expected Terraform Deliverables

- main.tf
- variables.tf
- outputs.tf
- terraform.tfvars
- Deployment documentation

Incident Response Report

- Incident summary: Order Service failed after its database hostname was changed to an invalid value.
- Impact assessment: users could log in and view products, but order creation was unavailable.
- Severity classification: SEV-2 because a core workflow was down while the whole platform was not fully unavailable.
- Detection: frontend order failure, container logs, Prometheus target status, and Grafana availability view.
- Root cause: DATABASE_URL pointed to wrong-postgres-host instead of postgres.
- Mitigation: restore the correct connection string and restart the Order Service.
- Resolution confirmation: health endpoint returned healthy, order creation worked, and monitoring returned to normal.

Timeline

10:00 Normal system operation.

10:05 Incident configuration applied.

10:07 Order creation failed in the UI.

10:10 Logs and monitoring confirmed Order Service degradation.

10:14 DATABASE_URL was corrected and service was restarted.

10:16 Health checks and order creation confirmed restoration.

Postmortem Analysis

The incident was caused by a configuration error. The system showed useful fault isolation because authentication, product, user, and chat services stayed available. The response process was effective because logs and monitoring quickly pointed to the affected service. The main improvement area is deployment validation before service startup.

Lessons Learned And Action Items

- Add environment variable validation during service startup.
- Add Prometheus alert rules for service downtime and error rate.
- Keep a short incident runbook with log, health check, and restart commands.
- Use environment templates to reduce manual deployment mistakes.
- Add automated smoke tests for login, product loading, order creation, and chat.

Supporting Evidence

Running Containers

- frontend: Up, port 80
- api-gateway: Up, port 8080
- auth, user, product, order, chat services: Up
- postgres, prometheus, grafana: Up
- internal Docker network: application-network

Supporting Evidence

Prometheus Targets

- auth-service target: UP
- user-service target: UP
- product-service target: UP
- order-service target: UP after restoration
- chat-service target: UP

Supporting Evidence

Grafana Dashboard

- Dashboard: Microservices Reliability Dashboard
- Panel: Requests Per Second
- Panel: Service Availability
- Datasource: Prometheus
- Restored services show availability value 1

Supporting Evidence

Incident Before And After

- Before: order-service database hostname was wrong-postgres-host
- Impact: order creation failed
- Analysis: container logs showed database connection error
- Mitigation: DATABASE_URL restored to postgres
- After: /orders/health returned healthy

Defense Preparation

- Explain the architecture from frontend to gateway, services, database, monitoring, and Terraform.
- Show docker compose ps and describe isolated containers.
- Demonstrate login, product display, order creation, and chat in the UI.
- Open Prometheus targets and explain the metrics endpoints.
- Open Grafana and explain request rate and availability panels.
- Run the incident override and show that the Order Service fails.
- Use logs to identify the wrong database hostname.
- Restore the correct configuration and verify service health.
- Explain Terraform resources and the public IP output.
- Finish with lessons learned and action items.

Conclusion

The project satisfies the assignment requirements by combining clean microservice code, Docker deployment, monitoring, Terraform infrastructure provisioning, realistic incident simulation, response documentation, postmortem analysis, and screenshot evidence.